



南京大學
NANJING UNIVERSITY

七、SPI控制器

主讲教师：闫浩
机器人与自动化学院



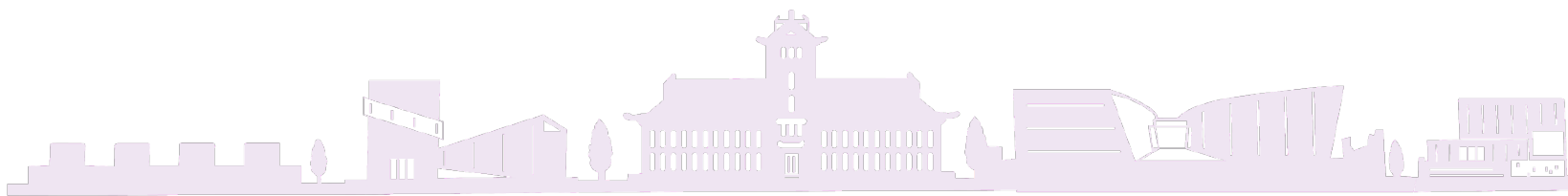
目录

CONTENTS

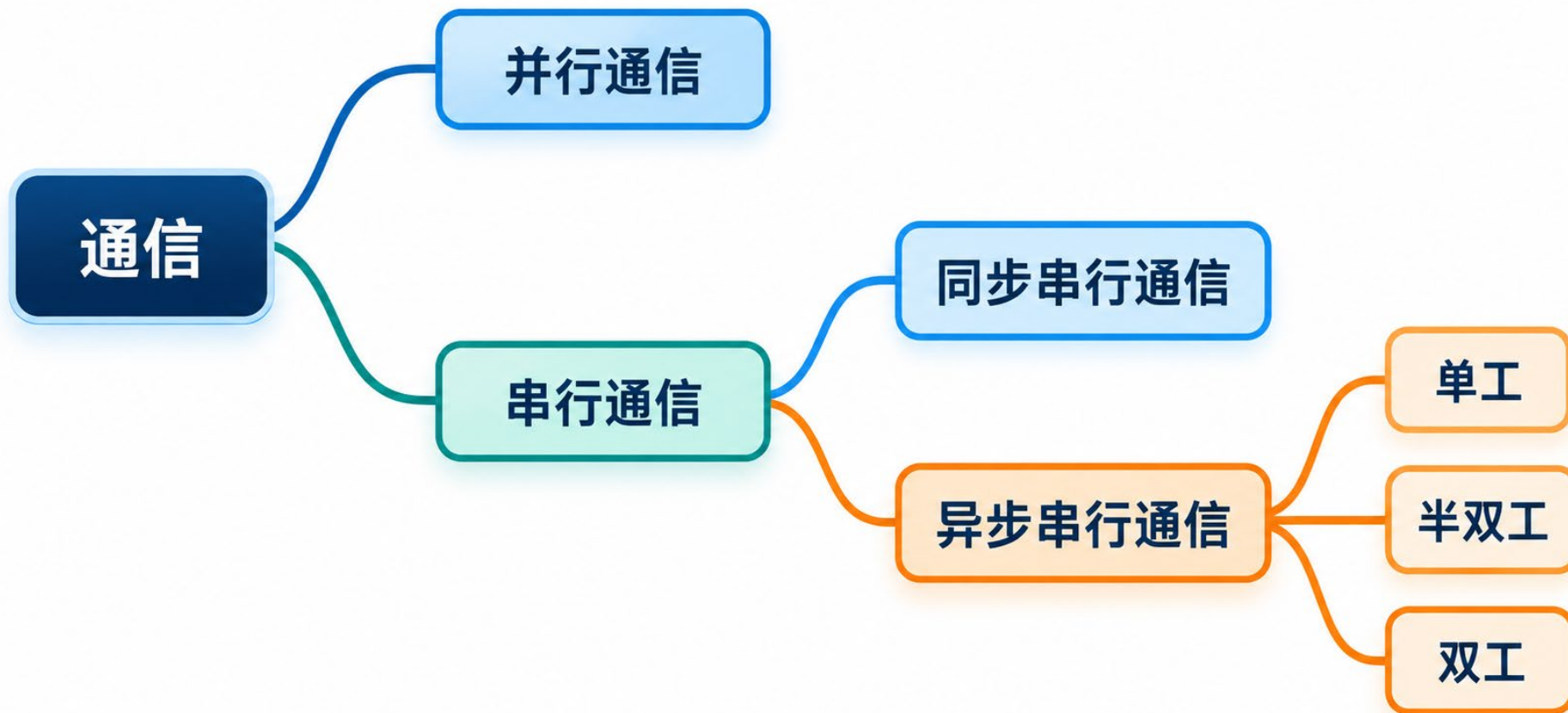
01 SPI协议简介

02 STM32的SPI工作原理

03 SPI应用步骤及常用库函数



01 SPI协议简介



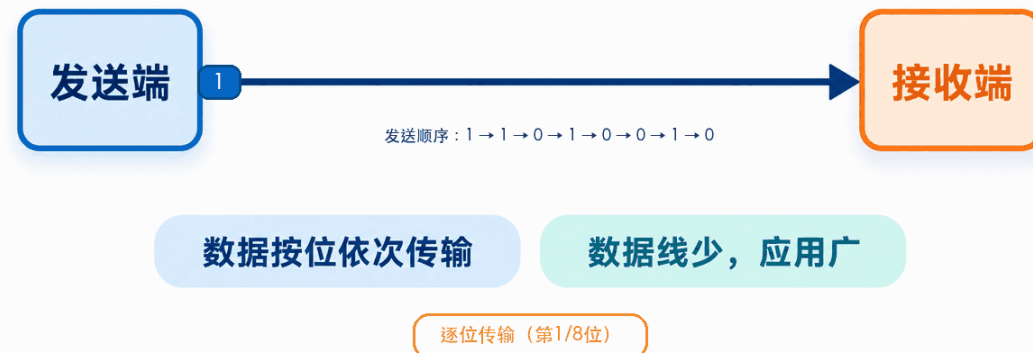
回顾：通信基本概念

并行通信



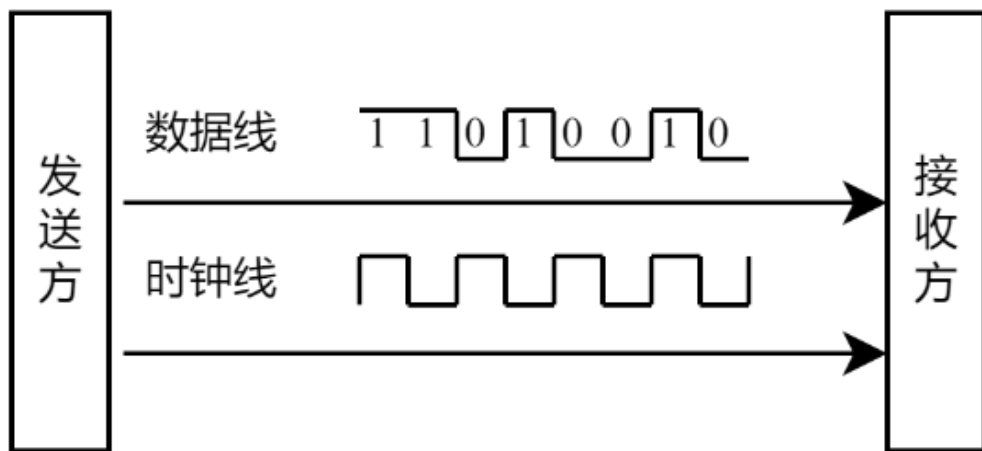
并行通信用多条数据线将数据字节中的各位**同时传输**。
效率高，成本高。

串行通信



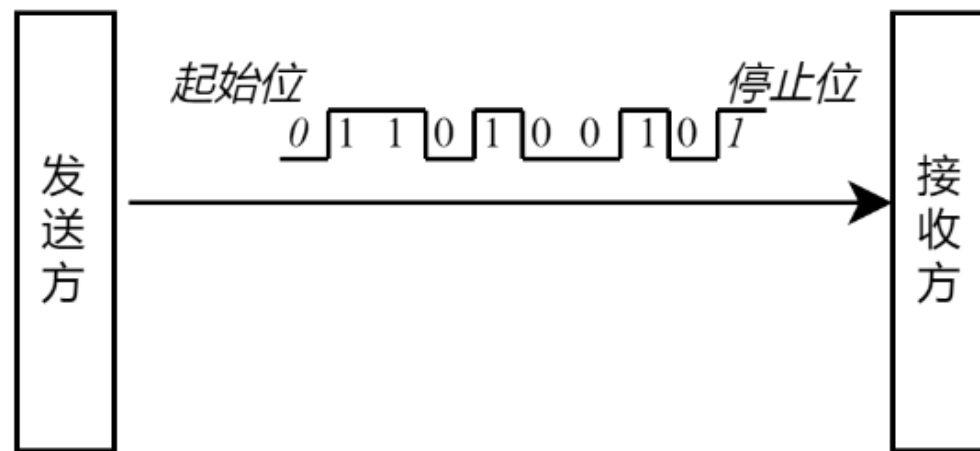
串行通信用一条数据线将数据字节中的各位**逐位传输**。
成本低，主流通信。

同步串行通信和异步串行通信



(a) 同步串行通信

有时钟线，传输信息帧（多个字符），用于芯片间通信。



(b) 异步串行通信

无时钟线，传输字符帧（一个字符），加起停标志，通信速率相同，用于设备通信。

按照数据传送方向，分为：

◆单工：

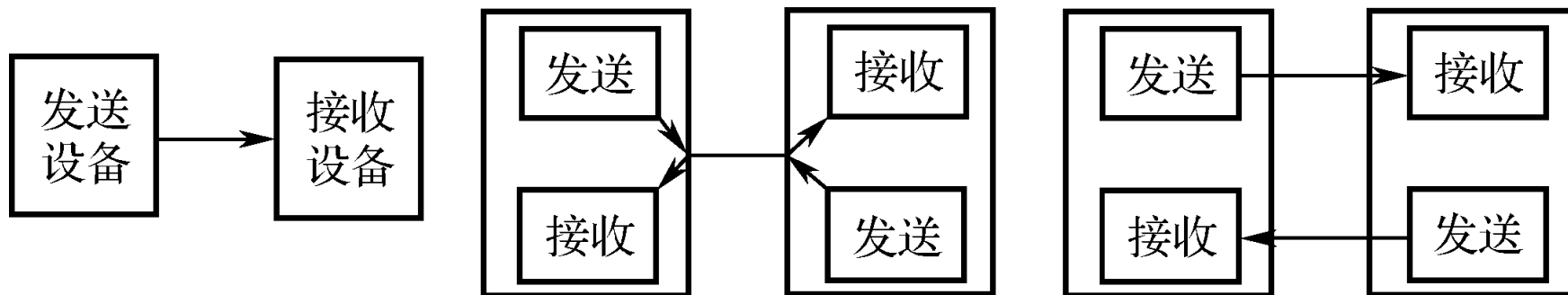
数据传输只支持数据在一个方向上传输；

◆半双工：

允许数据在两个方向上传输，但是，在某一时刻，只允许数据在一个方向上传输，它实际上是一种切换方向的单工通信；

◆全双工：

允许数据同时在两个方向上传输，因此，全双工通信是两个单工通信方式的结合，它要求发送设备和接收设备都有独立的接收和发送能力。



(a) 单工通信

(b) 半双工通信

(c) 全双工通信

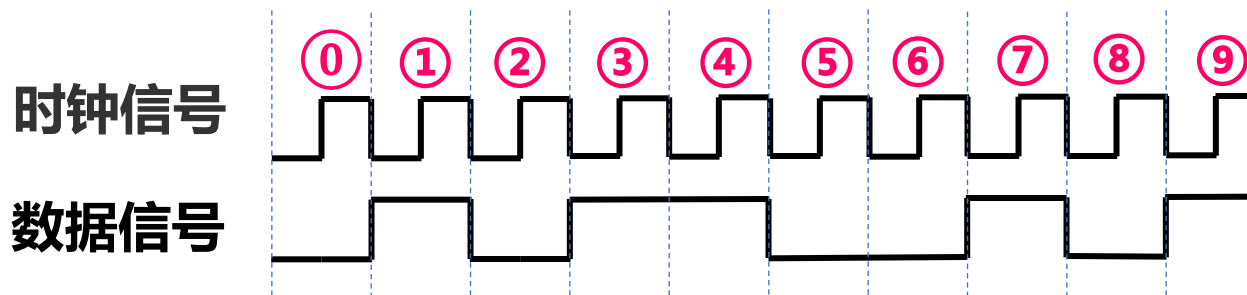
同步串行通信和异步串行通信

通信标准	引脚说明	通信方式	通信方向
UART (通用异步收发器)	TXD:发送端 RXD:接受端 GND:公共地	异步通信	全双工
USB	D+、D-	同步通信	半双工
SPI	SCK:同步时钟 MISO:主机输入，从机输出 MOSI:主机输出，从机输入	同步通信	全双工
I2C	SCL:同步时钟 SDA:数据输入/输出端	同步通信	半双工

同步串行通信和异步串行通信

1、同步通信：带时钟同步信号传输。

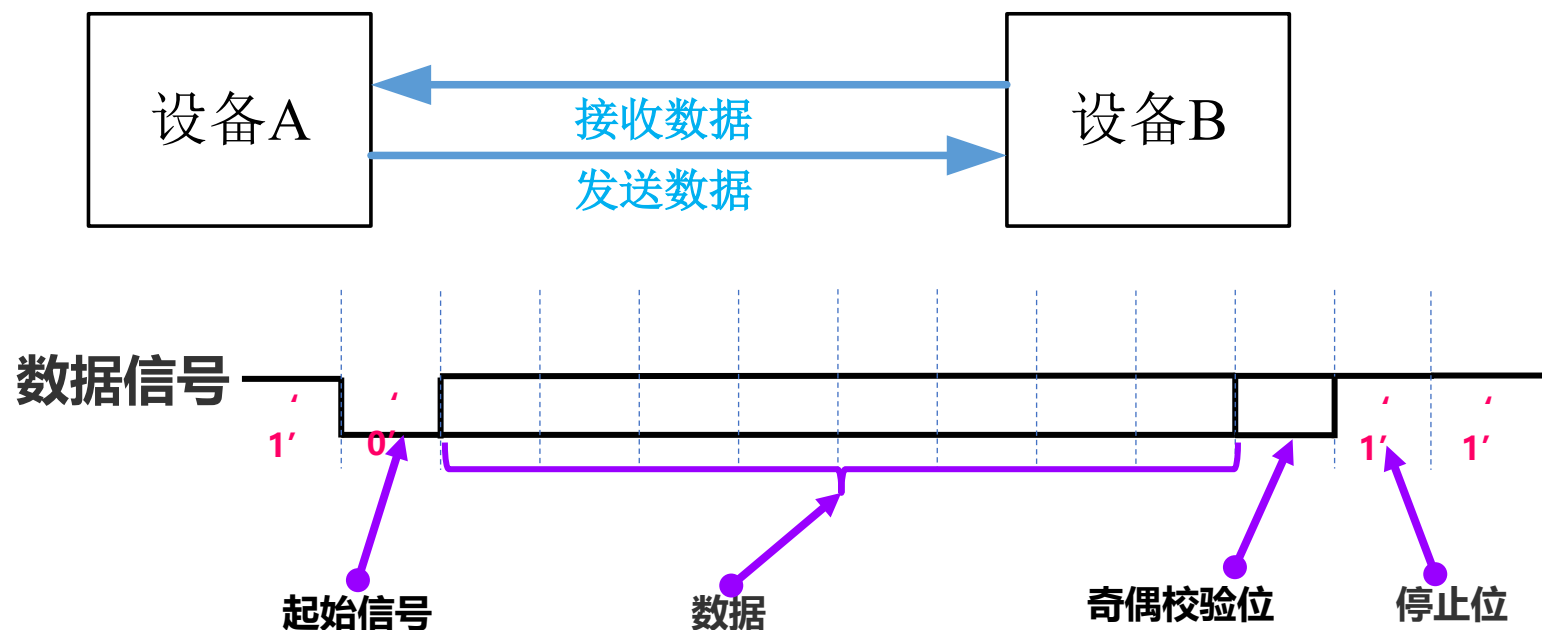
SPI, IIC通信接口, USB



同步串行通信和异步串行通信

2、异步通信：不带时钟同步信号。

UART(通用异步收发器)



串行外设接口 (Serial Peripheral Interface, SPI)

- 1、SPI是Motorola首先提出的**全双工**四线同步串行外围接口，采用**主从模式** (Master-Slave) 架构。支持单主多从模式应用，时钟由Master控制，在时钟移位脉冲下，数据按位传输，高位在前，低位在后 (MSB first) 。
- 2、4线SPI器件有**四个信号**：时钟(SPI CLK, SCLK)、主机输出从机输入(MOSI)、主机输入从机输出(MISO)、片选(SS/CS)。
- 3、产生时钟信号的器件称为主机。主机和从机之间传输的数据与主机产生的时钟同步。

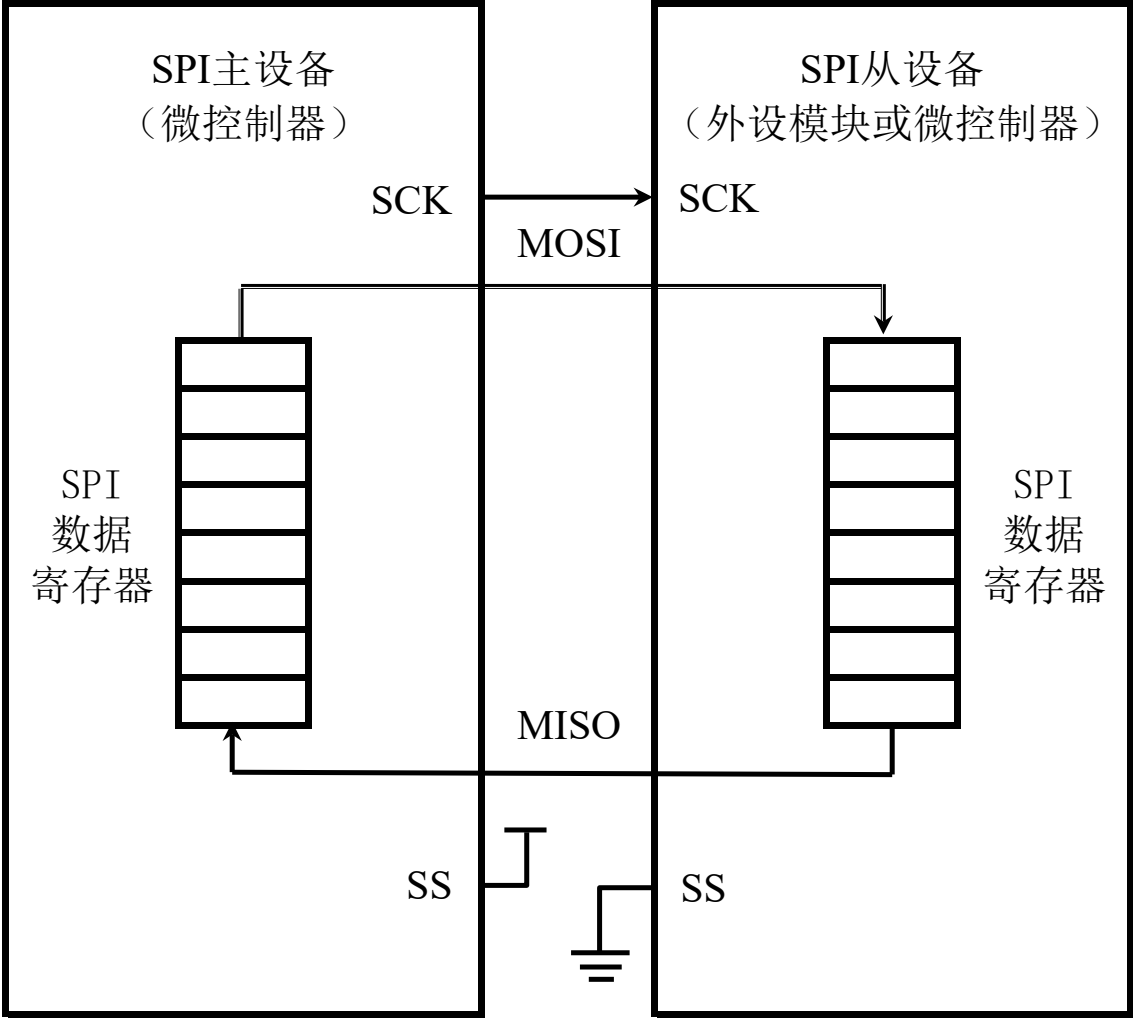
SPI在物理层体现为4条信号线，分别是SCK、MOSI、MISO和 (N) SS

1) SCK (Serial Clock) , 即时钟线，由主设备产生。不同的设备支持的时钟频率不同。

2) MOSI (Master Output Slave Input) , 即主设备数据输出 / 从设备数据输入线。

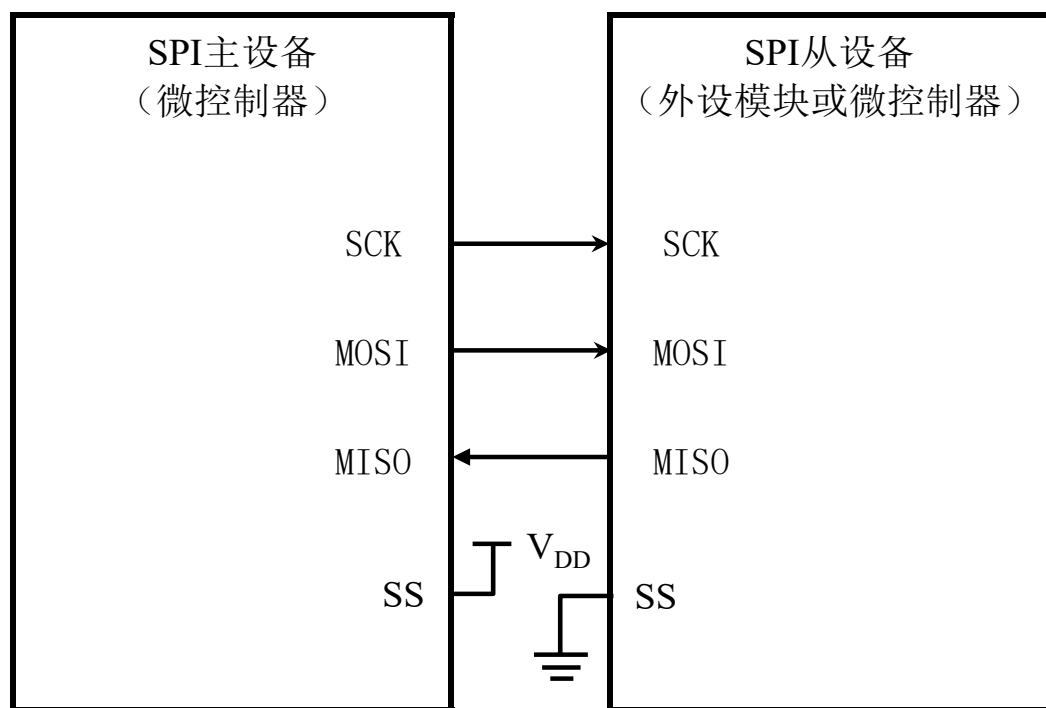
3) MISO (Master Input Slave Output) , 即主设备数据输入 / 从设备数据输出线。

4) SS (Slave Select) , 有时候也叫CS (Chip Select) , SPI从设备选择信号线，当有多个SPI从设备与SPI主设备相连（即一主多从）时，SS用来选择激活指定的从设备，由SPI主设备（通常是微控制器）驱动，低电平有效。当只有一个SPI从设备与SPI主设备相连（即一主一从）时，SS并不是必需的。因此，SPI也被称为三线同步通信接口。

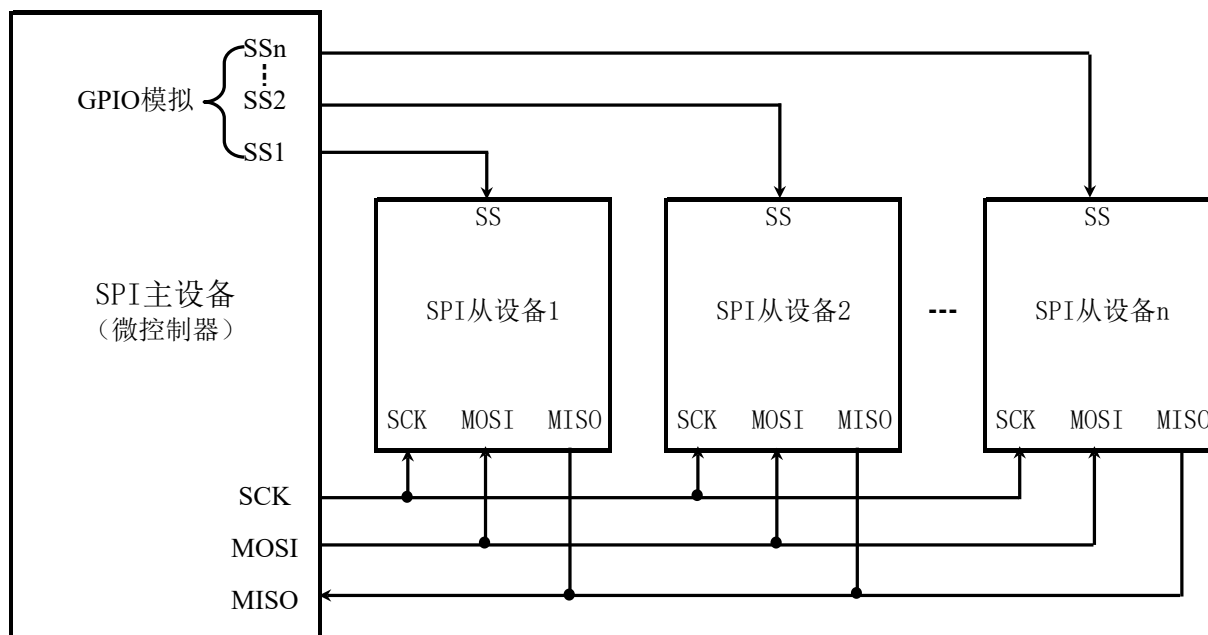


SPI组成

SPI互连主要有一主一从和一主多从两种互连方式

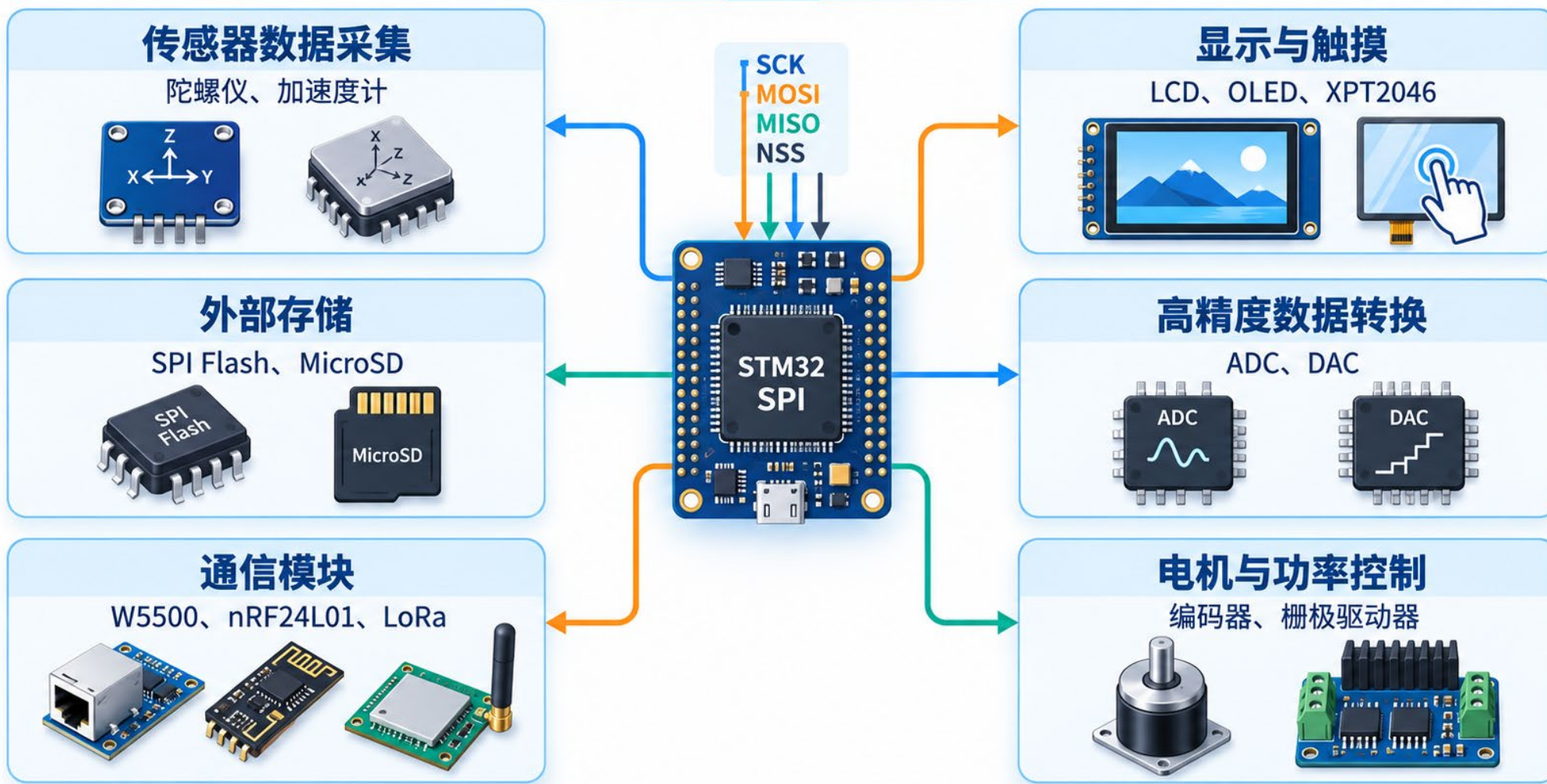


一主一从的SPI互连

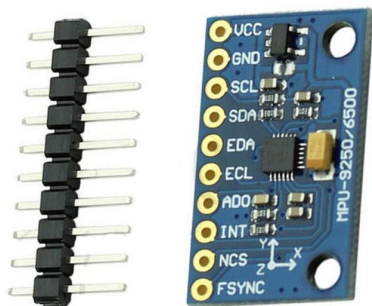


一主多从的SPI互连

STM32 SPI典型应用



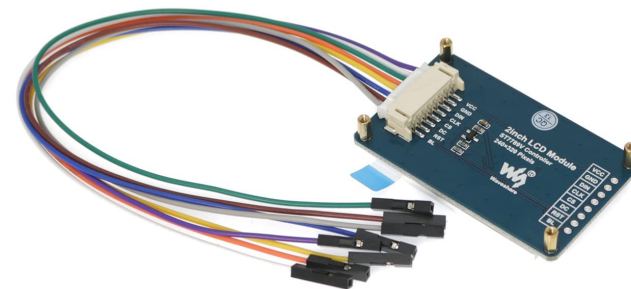
举例：几种以SPI通讯的器件或传感器



- **MPU6500**：读取加速度和角速度



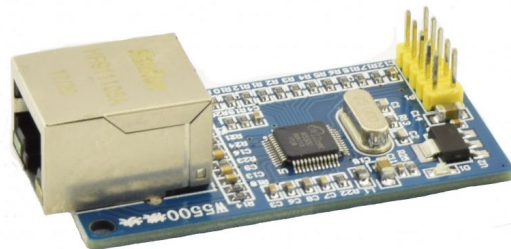
- **W25Q64**：练习页写入、扇区擦除和数据读取



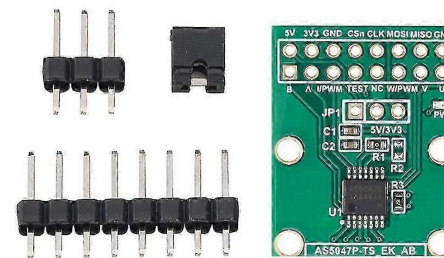
- **ST7789**：显示图形和文字



- **MCP3008**：采集模拟电压



- **W5500**：实现网络通信



- **AS5047P**：读取电机转子位置

02 STM32的SPI工作原理

STM32F4内部有6个SPI控制器，可与外部器件进行半双工/全双工的同步串行通信。

SPI控制器主要有以下特性：

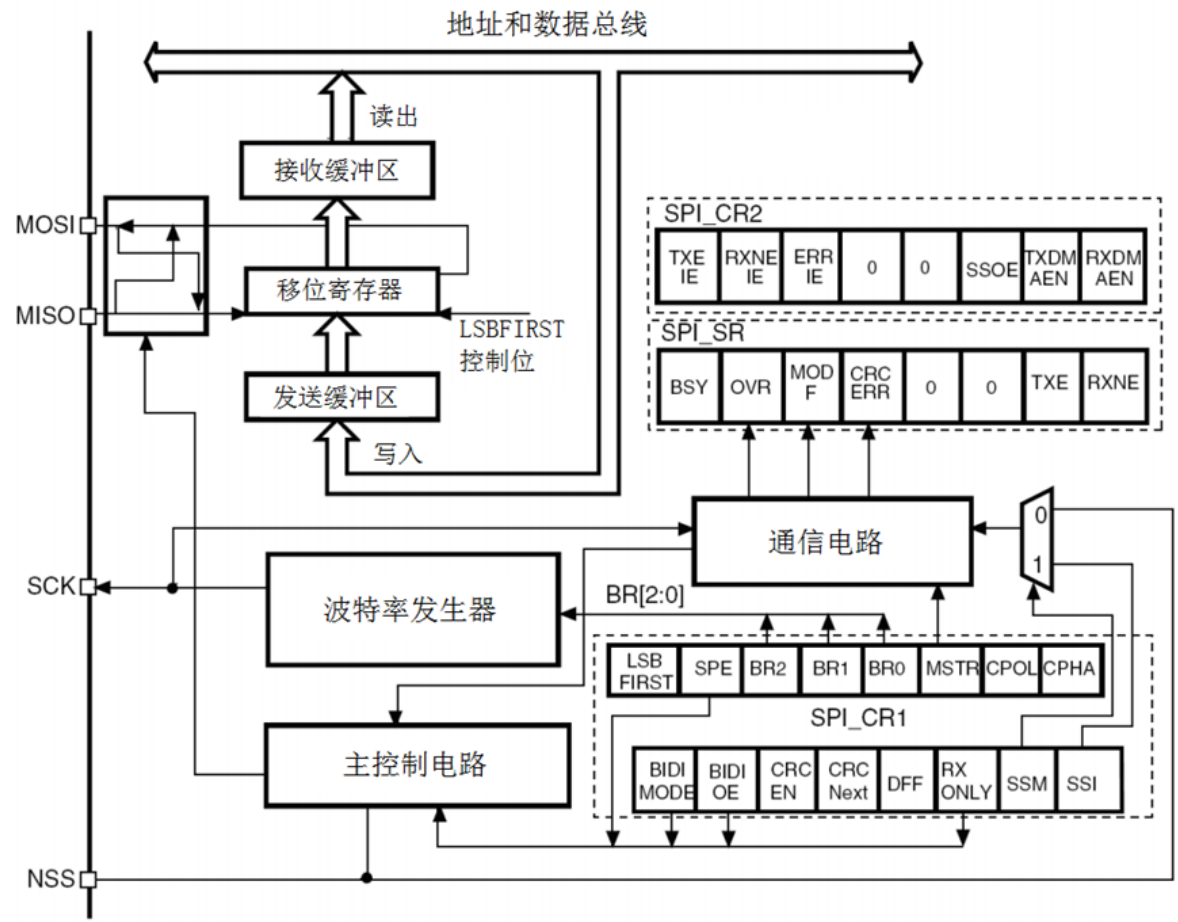
- 全双工同步传输；
- 8位或 16 位传输帧格式选择；
- 支持最高的SCK时钟频率为 $f_{pclk}/2$
- 主模式或从模式操作、多主模式功能；
- 可编程的时钟极性和相位；
- 可编程的数据顺序，先移位 MSB 或 LSB；
- 可触发中断的专用发送和接收标志；
- 具有 DMA 功能（直接存储器访问）

STM32 芯片有多个 SPI 外设，它们的 SPI 通讯引脚（MOSI、MISO、SCLK 及 NSS）通过 GPIO 引脚复用映射实现。

	SPI1	SPI2	SPI3	SPI4	SPI5	SPI6
总线	APB2	APB1	APB1	APB2	APB2	APB2
MOSI	PA7/PB5	PB15/PC3/PI3	PB5/PC12/PD6	PE6/PE14	PF9/PF11	PG14
MISO	PA6/PB4	PB14/PC2/PI2	PB4/PC11	PE5/PE13	PF8/PH7	PG12
SCLK	PA5/PB3	PB10/PB13/PD3	PB3/PC10	PE2/PE12	PF7/PH6	PG13
NSS	PA4/PA15	PB9/PB12/PI0	PA4/PA15	PE4/PE11	PF6/PH5	PG8

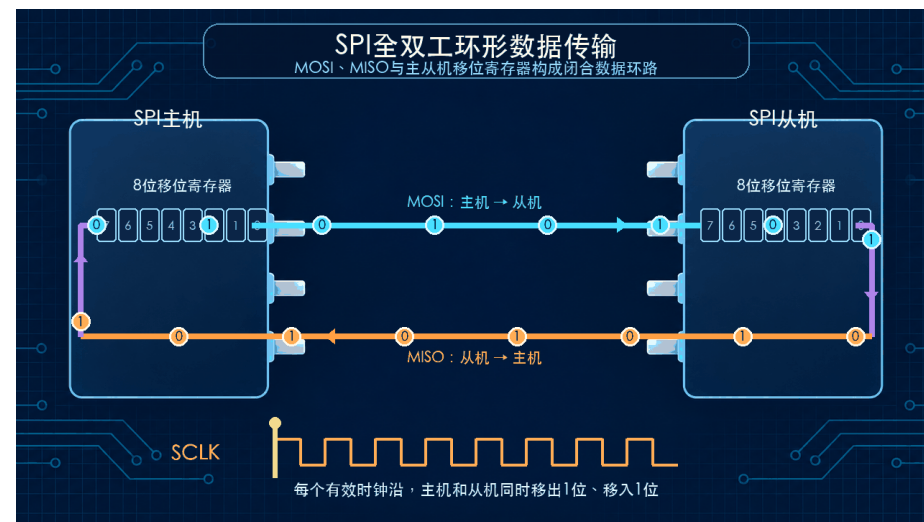
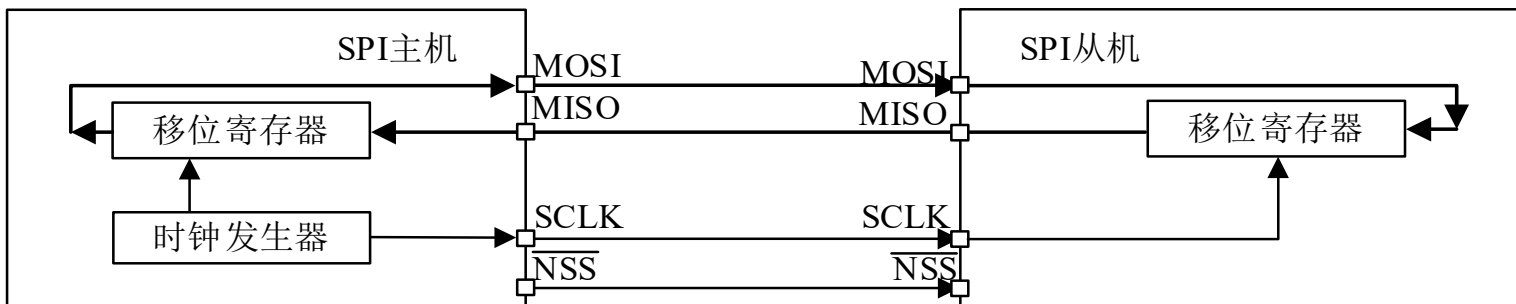
6个SPI控制器中**SPI1、SPI4、SPI5、SPI6** 挂载在 **APB2**总线上，
最高通信速率达**45Mbtis/s**，**SPI2、SPI3**挂载在 **APB1** 总线上，
最高通信速率为 **22.5Mbits/s**。

STM32F微控制器SPI主要由**波特率发生器**、**收发控制**和**数据存储转移**三部分组成。波特率发生器用来产生SPI的SCK时钟信号，收发控制主要由控制寄存器组成，数据存储转移主要由移位寄存器、接收缓冲区和发送缓冲区等构成。



1 SPI数据传输

在MOSI、MISO和SPI主从机内部的数据寄存器构成一个数据串行传输的环路，在时钟SCLK的控制下实现数据的环形传输。



要开始SPI通信，主机必须发送时钟信号，并通过使能NSS信号选择从机。

2 时钟信号的相位和极性

SPI_CR寄存器的**CPOL**和**CPHA**位，能够组合成**四种**可能的时序关系。

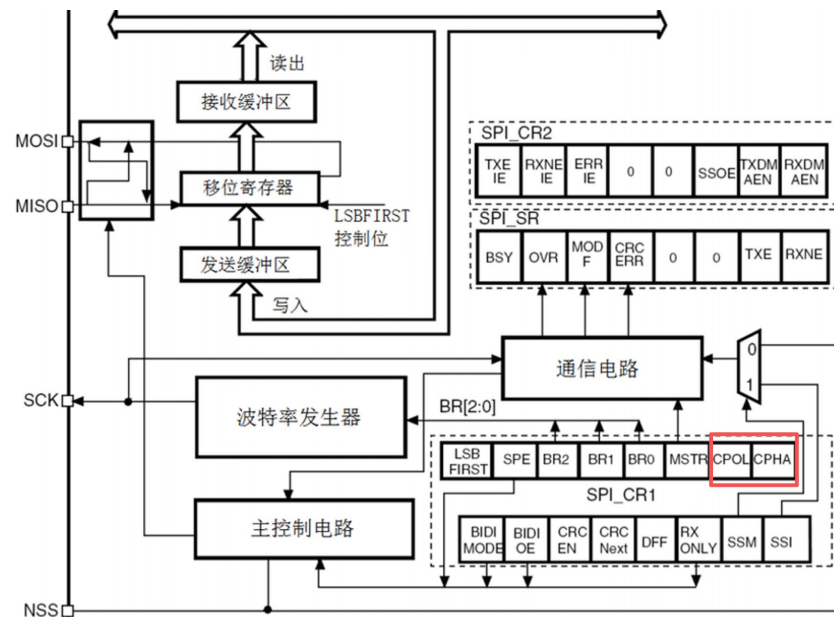
1、CPOL极性(Clock Polarity,CPOL)

SPI的CPOL，表示当总线空闲时，SCLK的极性，其电平的值是低电平0还是高电平1：

- CPOL=0，时钟**空闲时候**的电平是低电平，所以当SCLK**有效的时候**，就是高电平；
- CPOL=1，时钟**空闲时候**的电平是高电平，所以当SCLK**有效的时候**，就是低电平。

2、CPHA相位(Clock Phase,CPHA)

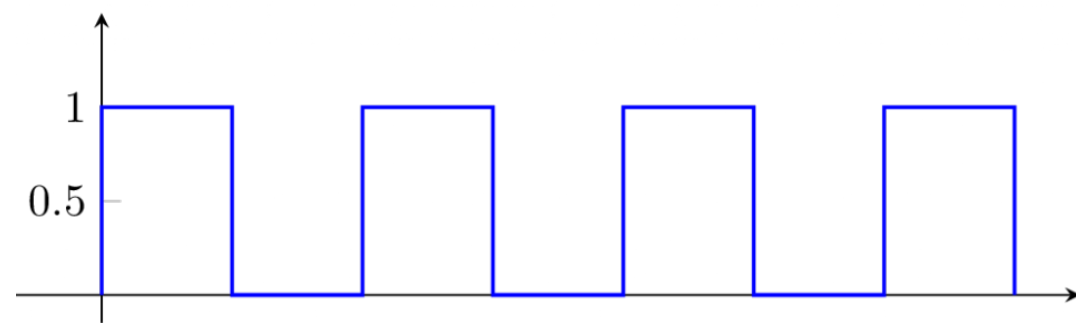
CPHA相位对应着数据采样是在第一个边沿（**空闲态**转电平切换到**相反电平**）还是第二个边沿，**0**对应着**第一个边沿**，**1**对应着**第二个边沿**。



3 四种SPI模式

主机必须根据从机的要求选择时钟极性和时钟相位。

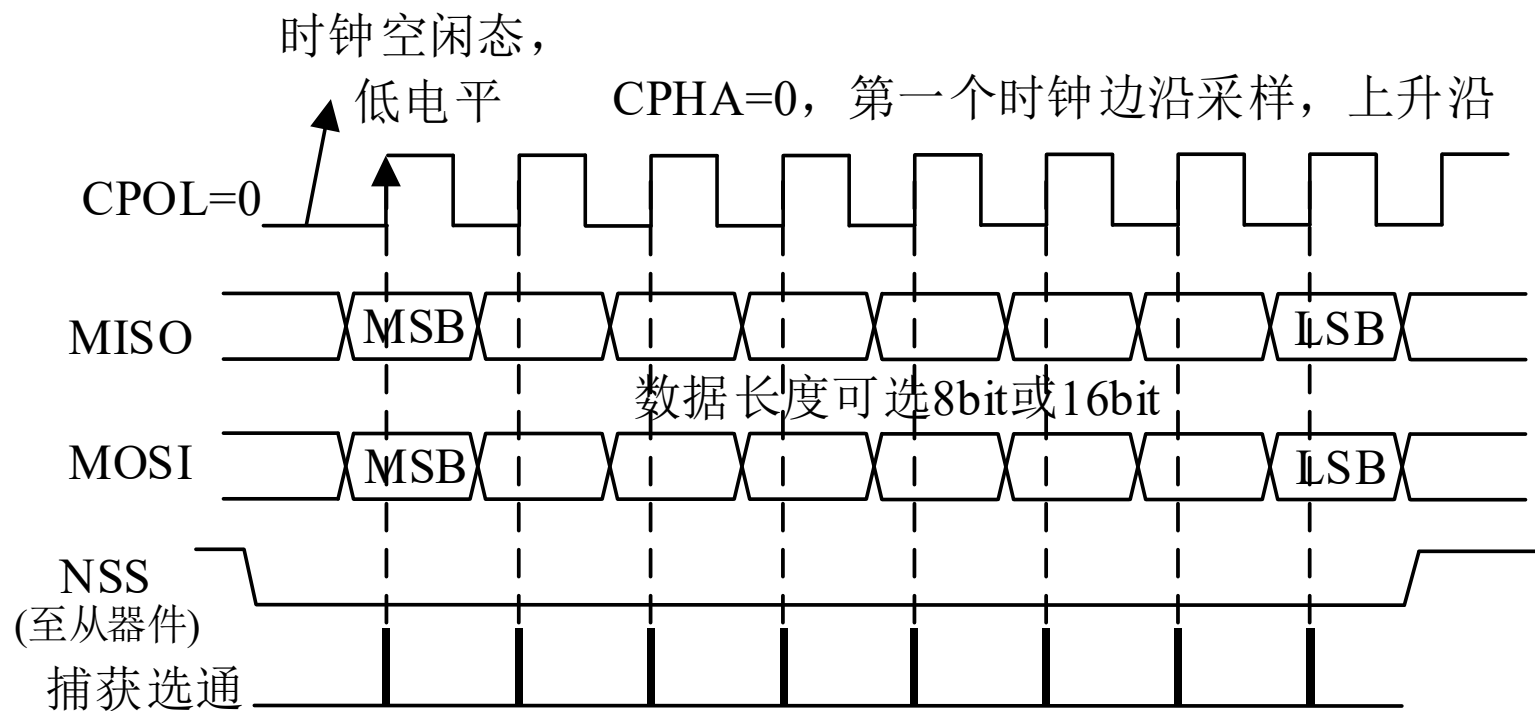
根据CPOL和CPHA位的选择，有四种SPI模式可用。



SPI 模式	CPOL	CPHA	空闲时时钟极性	采样/移位数据的时钟相位
0	0	0	低电平	第一个边沿，数据在上升沿采样，在下降沿移出
1	0	1	低电平	第二个边沿，数据在下降沿采样，在上升沿移出
2	1	0	高电平	第一个边沿，数据在下降沿采样，在上升沿移出
3	1	1	高电平	第二个边沿，数据在上升沿采样，在下降沿移出

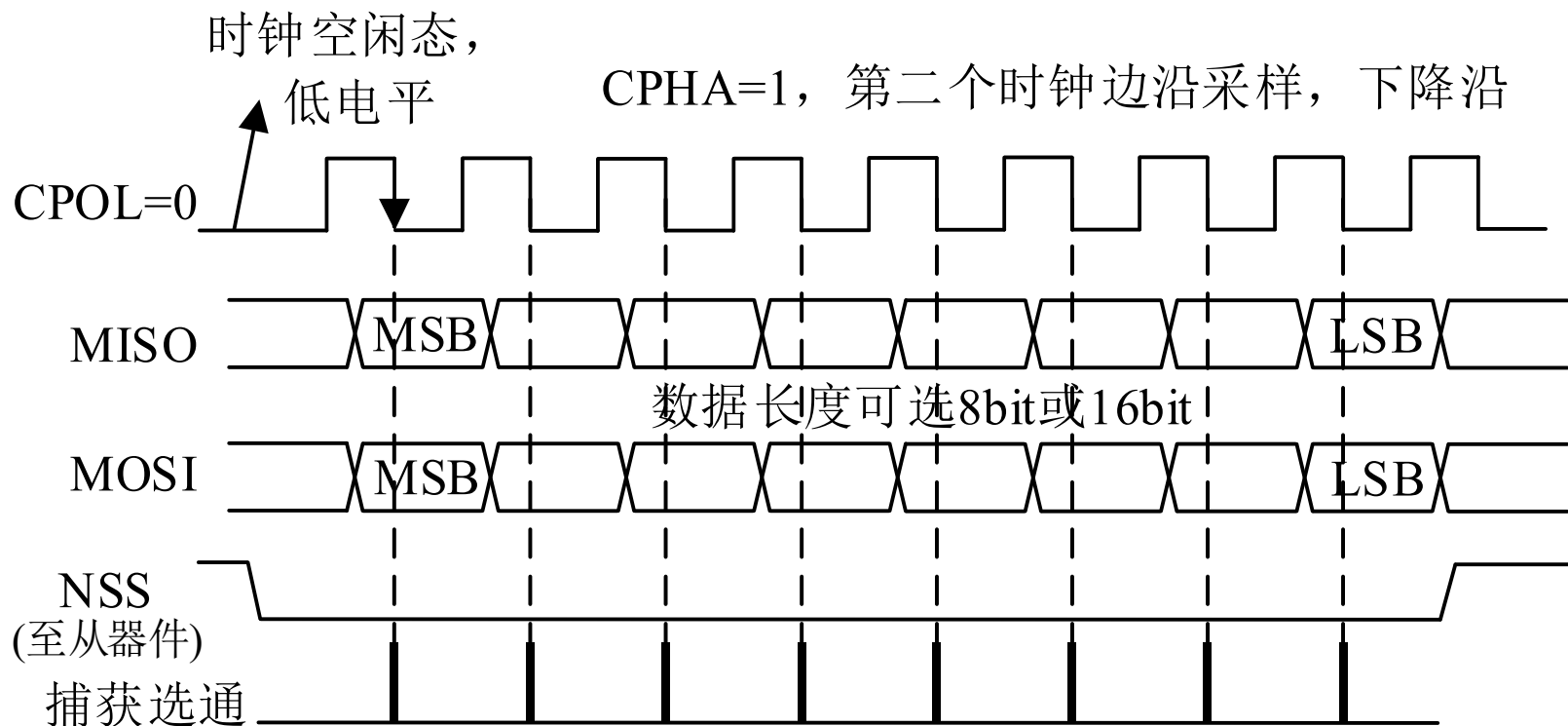
3 四种SPI模式

- 1、模式0：在此模式下，时钟极性CPOL=0，表示时钟信号的**空闲状态为低电平**。时钟相位CPHA=0，数据在第一个边沿（上升沿）采样，并且数据在时钟信号接下来的下降沿移出。



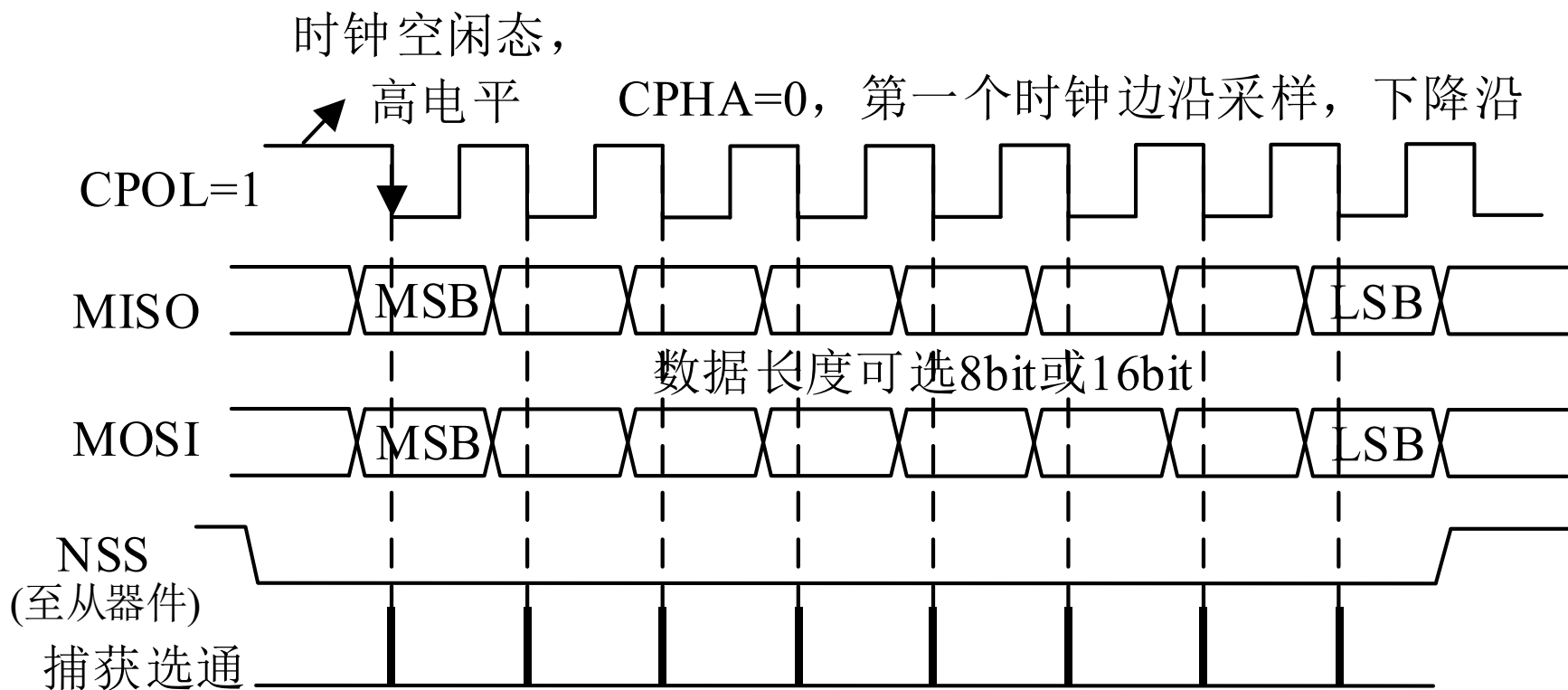
3 四种SPI模式

2、模式1：在此模式下，时钟极性CPOL=0，表示时钟信号的空闲状态为低电平。时钟相位CPHA=1，数据在第二个边沿（下降沿）采样，并且数据在时钟信号接下来的上升沿移出。



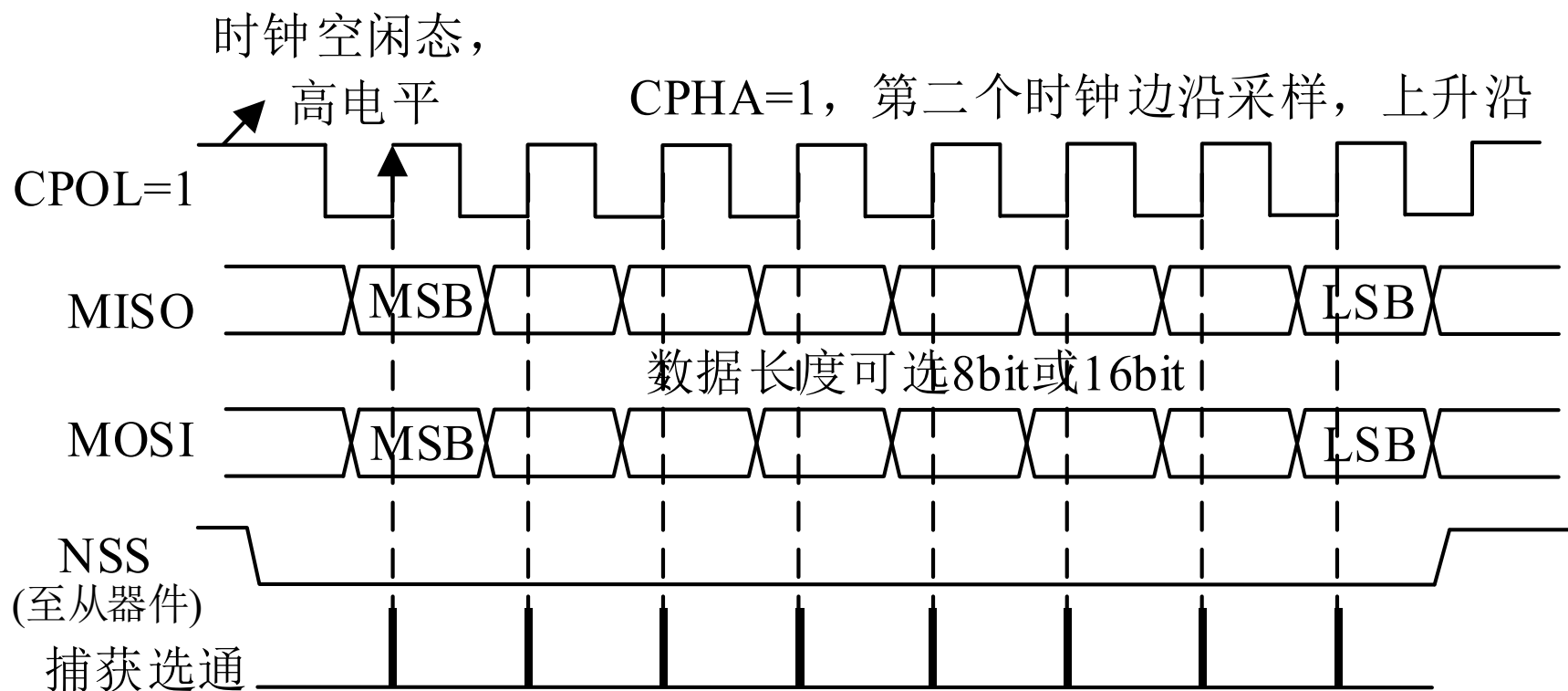
3 四种SPI模式

3、模式2：在此模式下，时钟极性CPOL=1，表示时钟信号的空闲状态为高电平。时钟相位CPHA=0，数据在第一个边沿（下降沿）采样，并且数据在时钟信号接下来的上升沿移出。



3 四种SPI模式

4、模式3：在此模式下，时钟极性CPOL=1，表示时钟信号的空闲状态为高电平。时钟相位CPHA=1，数据在第二个边沿（上升沿）采样，并且数据在时钟信号接下来的下降沿移出。



03 SPI应用步骤及常用库函数

1 SPI典型应用步骤

以SPI5工作在全双工主模式为例，使用PF6、PF7、PF8、PF9分别作为作为SPI的NSS、SCLK、MISO和MOSI的复用引脚。

	SPI1	SPI2	SPI3	SPI4	SPI5	SPI6
总线	APB2	APB1	APB1	APB2	APB2	APB2
MOSI	PA7/PB5	PB15/PC3/PI3	PB5/PC12/PD6	PE6/PE14	PF9/PF11	PG14
MISO	PA6/PB4	PB14/PC2/PI2	PB4/PC11	PE5/PE13	PF8/PH7	PG12
SCLK	PA5/PB3	PB10/PB13/PD3	PB3/PC10	PE2/PE12	PF7/PH6	PG13
NSS	PA4/PA15	PB9/PB12/PI0	PA4/PA15	PE4/PE11	PF6/PH5	PG8

1、开启SPI5控制器时钟和通信线复用引脚端口GPIOF的时钟。

使能SPI5时钟：

```
RCC_APB2PeriphClockCmd (RCC_APB2Periph_SPI5, ENABLE);
```

使能GPIOA时钟：

```
RCC_AHB1PeriphClockCmd (RCC_AHB1Periph_GPIOA, ENABLE);
```

SPI5

APB2

PF9/PF11

PF8/PH7

PF7/PH6

PF6/PH5

2、初始化引脚

复用PF6~PF9到SPI5：

```
GPIO_PinAFConfig(GPIOF,GPIO_PinSource6,GPIO_AF_SPI5);
```

```
GPIO_PinAFConfig(GPIOF,GPIO_PinSource7,GPIO_AF_SPI5);
```

```
GPIO_PinAFConfig(GPIOF,GPIO_PinSource8,GPIO_AF_SPI5);
```

```
GPIO_PinAFConfig(GPIOF,GPIO_PinSource9,GPIO_AF_SPI5);
```

将引脚设置为复用模式，并初始化：

```
GPIO_Init(GPIOF, &GPIO_InitStructure);
```

3、初始化SPI5控制器工作模式。

```
SPI_Init(SPI5, &SPI_InitStructure);
```

4、使能SPI5控制器

```
SPI_Cmd(SPI5, ENABLE);
```

5、中断使能

如果需要使用中断，需要配置NVIC和使能相应的SPI5中断事件。

SPI5

APB2

PF9/PF11

PF8/PH7

PF7/PH6

PF6/PH5

2 常用库函数

头文件: stm32f4xx_spi.h

源文件: stm32f4xx_spi.c

1、SPI初始化函数

`void SPI_Init(SPI_TypeDef* SPIx, SPI_InitTypeDef* SPI_InitStruct);`

1)、参数1: SPI_TypeDef* SPIx, SPI应用对象, 是一个结构体指针

```
#define SPI1 ((SPI_TypeDef *) SPI1_BASE)
```

2)、参数2: SPI_InitTypeDef* SPI_InitStruct, 是SPI应用对象初始化结构体指针

```
typedef struct
{
    uint16_t SPI_Direction; //定义SPI单向还是双向传输
    uint16_t SPI_Mode;      //定义SPI的主从模式
    uint16_t SPI_DataSize;   //定义SPI传输数据宽度
    uint16_t SPI_CPOL;       // SPI的CPOL极性, 定义当SPI空闲时, SCLK的极性
    uint16_t SPI_CPHA;       // SPI的CPHA 相位
    uint16_t SPI_NSS;        //NSS引脚管理方式
    uint16_t SPI_BaudRatePrescaler; //时钟频率
    uint16_t SPI_FirstBit;   //最先输出的数据位
    uint16_t SPI_CRCPolynomial; //CRC多项式
}SPI_InitTypeDef;
```

2 常用库函数

头文件: stm32f4xx_spi.h

源文件: stm32f4xx_spi.c

```
/* FLASH_SPI 模式配置 */
// FLASH芯片 支持SPI模式0及模式3, 据此设置CPOL CPHA
SPI_InitStructure.SPI_Direction = SPI_Direction_2Lines_FullDuplex;
SPI_InitStructure.SPI_Mode = SPI_Mode_Master;//主机模式
SPI_InitStructure.SPI_DataSize = SPI_DataSize_8b;//数据位=8bit
SPI_InitStructure.SPI_CPOL = SPI_CPOL_High;//SCK 引脚在 空闲状态处于高电平
SPI_InitStructure.SPI_CPHA = SPI_CPHA_2Edge;//SCK第二个边沿采样数据线
SPI_InitStructure.SPI_NSS = SPI_NSS_Soft;//内部 从器件管理
SPI_InitStructure.SPI_BaudRatePrescaler = SPI_BaudRatePrescaler_2;//波特
率预分频器
SPI_InitStructure.SPI_FirstBit = SPI_FirstBit_MSB;//先发送数据高位
SPI_InitStructure.SPI_CRCPolynomial = 7;//
SPI_Init(SPI5, &SPI_InitStructure);
```

2 常用库函数

头文件: `stm32f4xx_spi.h`

源文件: `stm32f4xx_spi.c`

2、SPI使能函数

```
void SPI_Cmd(SPI_TypeDef* SPIx, FunctionalState NewState);
```

3、SPI发送数据函数

```
void SPI_I2S_SendData(SPI_TypeDef* SPIx, uint16_t Data);
```

4、SPI接收数据函数

```
uint16_t SPI_I2S_ReceiveData(SPI_TypeDef* SPIx);
```

5、SPI检测状态标志函数

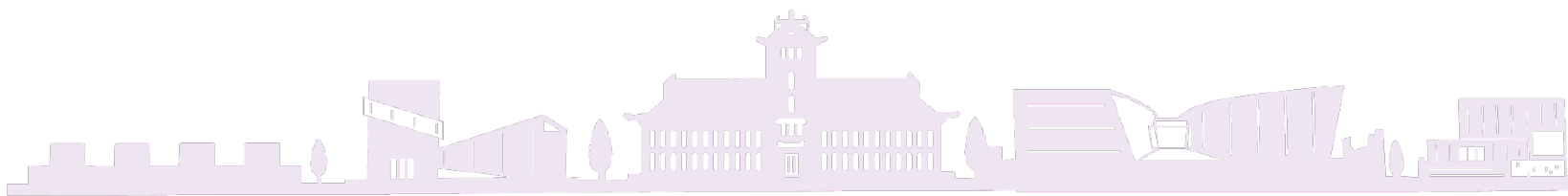
```
FlagStatus SPI_I2S_GetFlagStatus(SPI_TypeDef* SPIx, uint16_t SPI_I2S_FLAG);
```

本章小结

01 SPI协议简介

02 STM32的SPI工作原理

03 SPI应用步骤及常用库函数



谢谢!